

9

Using React Query for Server-Side-Driven State Management

Welcome, my dear reader, to the last chapter describing state management solutions for our Funbook app. In the previous chapter, we looked at the youngest state management library (as of the writing of this book) – **Jotai**. Jotai is a minimal solution, based on ideas proposed by the **Facebook** team in their open source library – **Recoil**. **React Query** is minimal as well but in a very different sense. React Query is created for managing fetching and mutating data on the server. In this chapter, we will look at what React Query has to offer. We will start by taking a broad look at this library; we will then implement it for data fetching. With our current app setup, we don't have a real backend server to communicate with, so we can only look at mutating data in theory. We will also look at a few special-

ized utilities created for **React Native** by the React Query team.

Here's a list of topics we will cover in this chapter:

- What is React Query and why is it in this book?
- Installing and configuring React Query
- Using React Query for data fetching
- Other React Query functionalities
- React Query utilities for React Native

By the end of this chapter, you will have a good understanding of how you can use React Query to improve your developer experience and your code bases. You will have a good knowledge of how to handle fetching data with React Query and a general knowledge of other functionalities of this library.

Technical requirements

In order to follow along with this chapter, you will need some knowledge of **JavaScript** and **ReactJS**. If you have followed at least *Chapters 1* through *4* of this book, you should be able to go forward without any issues.

Feel free to use an IDE of your choice, as **React Native** does not need any specific functionality. Currently, the most popular

IDEs for frontend developers are Microsoft's VSCode, Atom, Sublime Text, and WebStorm.

The code snippets provided in this chapter are here to illustrate what we should be doing with the code. They do not provide the whole picture. For a better experience while coding alongside reading this chapter, please open the GitHub repo in your IDE and look at the files in there. You can either start with the files in the folder named **example-app-full** or **chapter-9**. If you start with **example-app-full** you will be responsible for implementing the solutions described in this chapter. If you choose to look at **chapter-9** you will see the entire solution implemented by me.

If you get stuck or lost, you can check the code in the GitHub repo: <https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native/tree/main/chapter-9>.

What is React Query and why is it in this book?

First things first: let's talk about the name of this library. In this chapter, I use the name React Query, it is also a commonly used name. However, the creator of React Query, *Tanner Linsley*, did some restructuring in 2022, in the open source li-

braries that he owns and maintains. He created an umbrella name, **TanStack**, and placed a plethora of libraries under this name. And so, React Query became TanStack Query, as of React Query version 4. You can find a link to the TanStack home page in the *Further reading* section at end of this chapter.

Now that we have the name out of the way, let's talk about the place of React Query in this book. React Query is *not* a state management library. It's a library offering a solution for comfortable fetching and data mutations on the server. Why are we talking about it then? Because it turns out that efficient communication with the server can replace any need for global state management. Given our real-life social media app clone, we've been managing liked images in every chapter. What if, instead of working with the app state, every time a user likes an image, we sent that information to the server? Or when the user visits the **FavoritedImages** surface we pull the latest version of the list from the server? You may think: "Boy, that would be a lot of requests! A lot of loading states and the app being useless..." And you would be right! Except if you use React Query. React Query not only facilitates data fetching, but it also manages cached values, refreshing values, background fetching, and much more.

Now that we have a theoretical understanding of what React Query is, we can get to coding. Let's play with this non-state-

management library.

Installing and configuring React Query

Installing this library is no different from any other dependency, we need to run an installation script. To do this using **npm**, enter the following:

```
$ npm i @tanstack/react-query
```

Or if you would prefer to use **yarn**, enter the following:

```
$ yarn add @tanstack/react-query
```

Once the library is installed, we will need to add some minimal boilerplate. We will need to let our app know that we're using React Query. We will need to use a special wrapper. Do you see where I'm going with this? Yes! We will use a provider as follows:

```
// App.js
import {
  QueryClient,
```

```
    QueryClientProvider,  
  } from '@tanstack/react-query'  
  //...  
  const queryClient = new QueryClient()  
  export default function App() {  
    //...  
    return (  
      <SafeAreaProvider>  
        <QueryClientProvider client={queryClient}>  
    //...  
      </QueryClientProvider>  
    </SafeAreaProvider>  
    );  
  }  
  //...
```

We will start by importing the necessary functions from React Query – **QueryClient** and **QueryClientProvider**. Then, we will create a new **QueryClient** function and pass it to **QueryClientProvider**. Our app is ready to use React Query functionalities instead of simple fetching.

This is a good moment to make sure the app is running correctly on your simulator or device.

Once you have made sure installing new dependencies did not break anything unexpected in your project, we will be ready to implement real data fetching with React Query in the next section.

Using React Query for data fetching

As you know, we need to fetch a few different pieces of data for our app. We will fetch a list of avatars, a list of images for the feed surface, a list of images for the **FavoritedImages** surface, and a list of conversations. We are free to add the React Query fetching wherever we like. For simple queries, we can simply use the **useQuery** hook provided by the library in our components. We can also write our own custom hooks, holding more logic or conditions. Let's start by looking at the simplest possible example: querying the server to check whether the user is logged in.

In order to use a React Query hook in the top-level component where we set up our navigation to display either the login screen or not, we will need to reorganize our code a little bit. We cannot have **QueryClientProvider** in the return statement of the same component trying to use a **useQuery** hook. Let's change the name of the main component from **App** to **AppWrapped** and let's add this new app component in the **App.js** file:

```
// App.js
export default function App() {
```

```
    return (  
      <QueryClientProvider client={queryClient}>  
        <AppWrapped />  
      </QueryClientProvider>  
    )  
  };  
};
```

Now, let's change the name of the main component from **App** to **AppWrapped**, and let's remove **QueryClientProvider** from the child component. Let me remind you that if you ever get lost in the code examples, you can take a look at the GitHub repo: <https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native/tree/main/chapter-9>.

Our **AppWrapped** component should be ready to use the **use-Query** hook. Make sure you start by importing it as follows:

```
// App.js  
import {  
  useQuery,  
  //...  
} from '@tanstack/react-query'  
//...  
const fetchLoginStatus = async () => {  
  const response = await fetch(requestBase +  
    "/loginState.json");  
  return response.json();  
}  
const AppWrapped = () => {
```

```
const { data } = useQuery(['loginState'],
  fetchLoginStatus);
//...
{!data?.loggedIn ? (
  <Stack.Screen name='Login' component={Login} />
    ) : (
      <>
        <Stack.Screen
          name='Home'
        />
      </>
    )
  }
//...
```

After you've imported the **useQuery** hook, you need to create a function responsible for fetching and awaiting data from the server. This is the **fetchLoginStatus** function, which we will pass to the **useQuery** hook. This function can be created in any file you would like. Once we have the fetching set up, we need to use the **useQuery** hook in the component. We pull in a de-structured object key data, where we check the **loggedInStatus** value.

OBJECT DESTRUCTURING

*Depending on how often you use modern JavaScript, you may have noticed the destructuring syntax, where the **const** keyword is followed by items in curly or square brackets. This syntax is called destructuring assignment and is used to unpack values from arrays (square brackets), objects, or properties (curly brackets).*

```
const { data } = objectWithADataItem is the same as const  
data = objectWithADataItem.data.
```

Now that we have seen a simple example, let's look at something slightly more complex and create a custom hook and a dependent query.

Fetching image data

Fetching image data could be just as simple as fetching the login state data; however, I would like to talk about something more complicated. So, we will artificially complicate our lives by making sure the images are fetched only after the user is logged in. We will start by creating a custom hook called **useCustomImageQuery** inside a newly created **queries** folder. Our custom hook will return a **useQuery** hook:

```
// src/queries/useCustomImageQuery  
import { useQuery } from "@tanstack/react-query";  
import { requestBase } from "../utils/constants";  
const getImages = async () => {  
  const response = await fetch(requestBase +  
    "/john_doe/likedImages.json");  
  return response.json();  
}  
export const useCustomImageQuery = () => {  
  const { data } = useQuery(['loginState']);
```

```
return useQuery(  
  ["imageList"],  
  getImages,  
  {  
    enabled: data?.loggedIn,  
  });  
};
```

We started by importing the necessary **useQuery** function and our utility **requestBase**. Next, we created our fetching function called **getImages**. This function fetches data from a given API endpoint and returns it. Finally, we created a custom hook called **useCustomImageQuery**. On the first line of the hook, we check the **loginState** query. It looks different than in **App.js** where we used it first, doesn't it? It has only one parameter: **loginState**. This parameter is called a **query key** in the React Query world and it is literally a key to unlocking the power of React Query. Using this key, you can access any and all previously fetched data; you could also invalidate it manually or mutate it. As for us, we only need to check the login status now, using this particular query key.

The **return** statement of our custom hook consists of a **use-Query** hook with three parameters. In the first place, we have the awesomely important query key, **imageList**. Next, we see the call to the fetching function. Last but not least, we have a configuration object holding a key called **enabled**. This key de-

terminates when the given query should be called. In our case, the query will be called when the result of the **loginStatus** query returns the value of **true**. We just successfully set up React Query to fetch images. All that is left is to display them. Let's go to the **ListOfFavorited** component where we will replace the context call with the following custom hook:

```
// src/components/ListOfFavorited.js
import { useCustomImageQuery } from "../queries/
  useCustomImageQuery";
//...
export const ListOfFavorites = ({ navigation }) => {
  const { data: queriedImages } = useCustomImageQuery();
  //...
  return (
  //...
    <FlatList
      data={ queriedImages }
  //...
```

If everything went according to plan, you should be able to run the application now and see a list of favorited images, which is pulled by React Query from the backend. If you run into any trouble, remember that the custom hook we created is just another function, and can be debugged as such. You can put **console.log** in the component, in the hook, or in the **getImage** function called by the hook.

Hopefully, you were able to set up everything smoothly. In this section, we practiced using React Query for fetching and displaying data. We leveraged ReactJS knowledge – because we created a custom hook – but React Query hooks can be set up in many ways. Given that our app has a fake backend that can only serve data, this is as far as we can go in practical usage of React Query. I invite you though, my dear reader, to continue reading and find out what other great functionalities this library holds.

Other React Query Functionalities

As stated above, we can't use React Query in our example app to mutate data on the server because our backend is not robust enough. In a real-life application, you would most probably use an API that accepts a **POST** request just as well as a **GET** request. In these cases, you would be able to change data with the help of React Query. In order to do so, we are provided with another specialized hook: **useMutation**. Here's what this hook would look like if we could use it for the favorited images:

```
const imageListMutation = useMutation(newImage => {  
  return fetch('/john_doe/likedImages ',
```

```
    {method: 'POST', body: newImage})  
  });
```

The preceding function is very simple. It wraps a **fetch** call in a React Query utility. This utility offers us a few things, such as the fact that it has the following states: **isIdle**, **isLoading**, **isError**, and **isSuccess**. We can check these states and update the view accordingly. We would use this mutation in **ImageDetailsmodal**:

```
// src/surfaces/ImageDetailsmodal.js  
//...  
export const ImageDetailsmodal = ({ navigation }) => {  
  const imageListMutation = useMutation(newImage => {  
    return fetch('/john_doe/likedImages ',  
      {method: 'POST', body: newImage})  
  });  
  //...  
  return (  
    //...  
    <Pressable  
      onPress={() => {  
        imageListMutation.mutate({route.params.imageItem  
          })  
      }}  
    >  
    {mutation.isLoading ? (  
      <Text>Loading...</Text>  
    ) : (  

```

```
        <Icons  
          //...  
        /> )  
      }  
    </Pressable>  
  //...
```

Let me reiterate: we are doing a dry run of sending data to the server because our app's backend cannot handle a **POST** request.

In the preceding code, we started by adding a React Query mutation function to **ImageDetailsModal**. We passed it into the **Pressable** component. Then, inside the **Pressable** component, we added a ternary operator to check whether the mutation is in a loading state. In case it is, we display a **Text** component saying **Loading....** This is a minimal example of how you can take advantage of mutation states. In a real-world app, you would probably check for **isSuccess** and **isError** as well and you would probably handle loading more gracefully.

This is all nice, but the way we implemented the mutation above we would still need to re-fetch the data traditionally to have the latest version in the **ListOfFavorites** component. Unless, we use the full power of React Query and update the cached version of data, fetched previously through the **useCus-**

tomImageQuery hook! Here's what we would need to change in the mutation:

```
const updateImges = () => {  
  return fetch('/john_doe/likedImages ',  
    {method: 'POST', body: newImage})  
}  
const imageListMutation = useMutation(updateImges, {  
  onSuccess: data => {  
    queryClient.setQueryData(['imageList'], data)  
  }  
})
```

In the preceding code snippet, we started by extracting the **fetch** function for better readability. We then add **onSuccess** logic to the mutation and we tell it to update the item marked by the **imageList** query key with the new data. Thanks to this strategy we will not have to manually update the **imageList** data every time a mutation occurs. You can read more about updating after mutation responses in the TanStack documentation, linked in the *Further reading* section.

We have covered the two most important aspects of React Query: fetching and mutating data. However, there's much more functionality to be taken advantage of in a real-life project. You can check the fetching status, just like we did with the example mutation. You can also do parallel queries for fetching data simultaneously. If you want to, you can set initial

data to fill your views before fetching is complete. It is also possible to pause or disable queries whenever you need. For large datasets, there is a special type of query, a paginated query, which will batch data into consumable chunks. In case your data is infinite, React Query provides utilities for infinite queries. Many big apps may take advantage of prefetching data on page load.

I encourage you, my dear reader, to read the React Query documentation to be able to grasp all the possible solutions it offers. I was surprised myself while using React Query by how many common problems are solved out of the box by this library.

React Query utilities for React Native

As we all know, React Native has its own quirks as compared to pure ReactJS. React Query doesn't leave managing those quirks to the developers, but rather steps up with some interesting solutions. For example, there's an **onlineManager** that can be added to React Native apps to have our apps reconnect when they are online. If we would like to refresh or refetch data when the app is focused, we can use React Query's **focusManager** together with React Native's **AppState**. In some cases, we

may want to refetch data when a specific screen in our app is focused, and React Query offers a solution for that use case as well. If you want to read about these utilities and how to use them in more detail, head over to the TanStack documentation at <https://tanstack.com/query/v4/docs/react-native>.

Summary

React Query is battle tested for scaling applications and can be a great solution for all sorts of projects. In this chapter, we installed it in the Funbook app and added it to the app. We didn't configure anything specific, as our project is small and didn't require any changes from the default configuration. We then looked at how a simple data fetching mechanism can be used for checking the login status of the user. Next, we created and used another, more complex, data-fetching hook with a dependency. We displayed the fetched data and then we took a tour of other React Query utilities. React Query is the last stop in our journey through the world of state management libraries for React Native apps. I hope you enjoyed the ride!

I invite you, my dear reader, to accompany me to the last chapter, where we will summarize everything we have learned on the topic of state management in React Native apps.

Further reading

- <https://tanstack.com/> – The TanStack home page.
- <https://tanstack.com/query/v4/docs/guides/updates-from-mutation-responses> – TanStack Query, *Updates from Mutation Responses*.

